InnerVoice: Thinking While Talking with a Last-Layer Disclosure Gate for Streaming Speech Assistants

Anonymous submission

Abstract

Reasoning language models think before they speak, so a voice assistant built on one is silent for thousands of tokens; speak-while-solving methods instead learn a disclosure policy that interleaves private reasoning with public statements, but existing recipes rely on special tokens or decode-time controllers and leave open how to train the policy with RL without breaking it. We propose InnerVoice, a minimal change to a pretrained decoder: the last layer's feed-forward block becomes two experts (think / answer) selected per token by a disclosure gate on the hidden state, with LoRA elsewhere. The gate's bit labels each readable token as private or voiced, so the model runs unchanged in standard inference engines, and interleaved training data are converted programmatically from existing datasets with an entailment alignment. We find that (i) after SFT, InnerVoice matches think-then-speak fine-tuning where reasoning does not matter and beats it where it does, with half the waiting; (ii) dense disclosure rewards collapse GRPO training in three characteristic ways, traceable to REINFORCE on a near-deterministic argmax gate; and (iii) a bounded reward paid only on correct rollouts, a KL anchor to the SFT policy and hash-reproducible sampled routing make RL stable. On AIME25 the resulting Qwen3-4B model rises from 0.14–0.19 to 0.40 (Average@8, 12k budget), above the base model's native thinking mode (0.34) while speaking its first content after about 470 tokens instead of 10,000.

Introduction

A voice assistant cannot hide a chain of thought behind a loading spinner. If the model reasons for ten thousand tokens before its first sentence, the user hears silence for minutes; if it skips reasoning, accuracy on anything non-trivial collapses (on AIME25 a direct-answer fine-tune of Qwen3-4B scores 0.04). The streaming setting therefore asks a question that the usual *think-then-speak* recipe never has to answer: *when* is it safe to say something, and *what* should be said while the rest of the reasoning is still in progress?

Wei et al. (2026) frame this as learning a *disclosure policy* ("Speak-while-Solving", SxS): the model interleaves private reasoning with public statements, each public statement must be entailed by the reasoning so far, and the policy is trained first by supervised fine-tuning on interleaved trajectories and then by reinforcement learning with correctness and timing rewards. Their evaluation protocol—accuracy on the public stream, plus token-index measures of how long the user waits—is the one we adopt. Xie et al. (2025) reach a similar conclusion from the RL side on Knights & Knaves and MuSiQue, and Srinivas et al. (2025) show, for voice agents, that even grounded filler spoken while a reasoner works makes the interaction feel responsive.

This paper asks how little has to change in a pretrained decoder for it to think while it talks, and what it takes to train the resulting disclosure gate with RL without breaking it. Our contributions:

- **Architecture.** InnerVoice replaces only the last decoder layer's feed-forward block with two experts (think / answer) chosen per token by a disclosure router, plus LoRA elsewhere. Both streams are readable text; the router's bit tells the client which tokens to voice. Because the routing decision is a pure function of the last-layer hidden state, the model runs in a standard inference engine: our vLLM plugin reproduces the reference decoder exactly (8/8 greedy sequences, 882/882 routing labels identical), and recovering the labels on the training side is a single forward pass.
- **Data without new annotation.** We convert four existing datasets into entailment-aligned interleaved trajectories: per-step search traces (Knights & Knaves), decomposed multi-hop QA (MuSiQue), voice-agent dialogues with "think-a-chunk / voice-it" structure (ConvFill), and long-form mathematical solutions whose private and reader-facing parts are aligned by numeric anchors (DeepMath-103K; also distilled from Qwen3-32B, GPT-OSS-120B and GPT-6 Astra).
- **A scaled-down replication of the SxS protocol** on 1.7B/4B models, with the additional playback-simulator metrics a speech product cares about (time-to-first-content, pause distribution, premature commitments). SFT-stage InnerVoice ties think-then-speak SFT on tasks where thinking does not help and beats it where it does, with roughly half the waiting.
- **An RL study of disclosure rewards** with engine-based rollouts. We document three collapse modes of GRPO-style training on the gate and identify their mechanism

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

(dense shaping terms dominate the sparse correctness signal; argmax routing gives REINFORCE on the gate no exploration and pushes uncertain logits across the decision boundary), and we give a stable recipe: a bounded disclosure bonus paid only on correct rollouts, soft overlong punishment, a KL anchor to the SFT policy on tokens *and* gate decisions, and hash-reproducible sampled routing. On AIME25 the recipe takes Qwen3-4B INNERVOICE from 0.14–0.19 to 0.40 while speaking from token $\sim$470.

We also report what did not work, including an earlier attempt to replace readable private reasoning with recurrent *latent* computation, which gave no accuracy over a direct-answer baseline at any recurrence depth (Appendix).

Related Work

Disclosure policies and interleaved reasoning. Wei et al. (2026) introduce the SxS problem, the entailment-based data construction, the ARI/ABO/AIRW metrics and a two-stage SFT+RL recipe on Qwen3-4B (AIME25: base 66.3, SxS SFT 38.3, SxS RL 80.0 at a 32k budget). Xie et al. (2025) train interleaved reasoning with RL on Knights & Knaves and MuSiQue and report latency and accuracy gains; we reuse their choice of datasets. Srinivas et al. (2025) (ConvFill) pair a small conversational model with a slow reasoner and release aligned "thoughts / response" turns with silence markers, which we use as disclosure-timing labels. Our architectural question—a per-token gate inside the network, implemented with a two-expert last layer—is orthogonal to these works and compatible with their rewards.

Reasoning models and RL. Qwen3 (Qwen Team 2025) exposes a native thinking mode that we use as the "think-then-speak" baseline. Our RL follows GRPO (Shao et al. 2024; DeepSeek-AI 2025) with the overlong handling of DAPO (Yu et al. 2025); the policy gradient on binary gate decisions is plain REINFORCE (Williams 1992). Mixture-of-experts layers (Fedus, Zoph, and Shazeer 2022) motivate the two-expert block, although our router is supervised rather than load-balanced.

Latent reasoning. Coconut (Hao et al. 2024) replaces reasoning tokens by continuous states. We tried recurrent latent computation first; it produced no gain on Qwen3 (Appendix), which motivated keeping the private stream as readable text that can be inspected and, in a product, logged.

InnerVoice

Architecture

Let $h_t \in \mathbb{R}^d$ be the normalised input to the feed-forward block of the last decoder layer at position t (Fig. 1). INNERVOICE replaces that block with

$$y_t = \begin{cases} \mathrm{FFN}_{\text{think}}(h_t) & r_t = 1 \\ \mathrm{FFN}_{\text{answer}}(h_t) & r_t = 0 \end{cases}, \quad p_t = \sigma\big(\mathrm{MLP}_\rho([h_t; b_t])\big), \tag{1}$$

where both experts are initialised from the pretrained block and fully fine-tuned, MLP_ρ is a two-layer router ($d{+}4 \to 256 \to 1$) and $b_t \in \mathbb{R}^4$ are optional playback-buffer features (remaining audio, think tokens in the current block, chunks emitted, per-token think cost; zero in all experiments reported here). The decision r_t determines the expert that produces the *next* token's logits, so $r_t{=}1$ means "token $t{+}1$ is private". All other layers receive LoRA (Hu et al. 2022) with rank 16. An additive type embedding e_{think} on private input tokens was part of the original design; the ablation in Table 1 shows it is unnecessary and the RL models drop it, which is also what makes the model a pure per-token function and therefore engine-compatible.

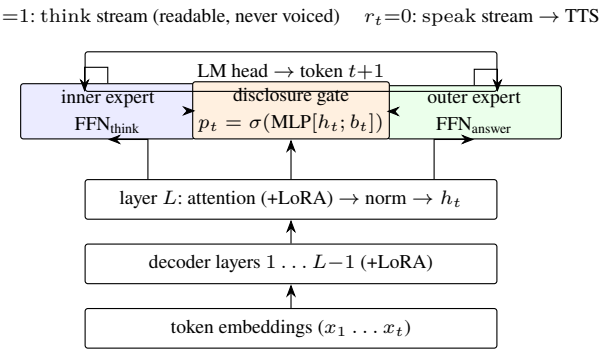


Figure 1: INNERVOICE: only the last layer's feed-forward block changes. The gate reads the last-layer hidden state h_t (and optional buffer features b_t) and selects the expert that produces token $t{+}1$; its bit labels that token as private or public. Both streams are ordinary text.

Training objective

Given a trajectory with token types $y_t \in \{\text{think}, \text{speak}\}$, the loss is the token cross-entropy over both streams plus a binary cross-entropy on the gate, $\mathcal{L} = \mathrm{CE}(\text{tokens}) + \lambda\,\mathrm{BCE}(p_t, \mathbb{1}[y_{t+1}{=}\text{think}])$, with positions where the type *switches* weighted $5\times$ and teacher-forced routing (the expert indicated by the label) during training. Gate accuracy at switch points is the quantity to watch: a trajectory with correct text but a gate that switches a token late voices the beginning of a private thought.

Streaming inference and the playback simulator

Decoding is one autoregressive loop. At each step the gate's bit decides which expert produced the logits and therefore the mode of the sampled token; speak tokens are appended to the current public chunk, which is sent to TTS at newlines; think tokens go to the private stream. No decode-time controller is needed. We also simulate playback with 0.03 s per generated token and 0.35 s per spoken word to report time-to-first-content (TTFC), the P50/P95 pause between spoken chunks, and the *premature-commitment* rate (public claims later contradicted or unsupported). Decode-time overrides of the gate ("think at most k tokens per block", "think only while audio is buffered") are evaluated in Table 2; they must be trained in, not imposed.

Interleaved trajectories from existing data

All training data are converted programmatically from public datasets; nothing is hand-annotated. **Knights & Knaves** (Xie et al. 2024) (6.2k puzzles, 2–8 people): the per-step search trace is the private stream; a person's verdict becomes a public chunk after the last reasoning step that mentions that person. **MuSiQue** (Trivedi et al. 2022) (6k questions, 2–4 hops): the decomposition with retrieved passages is private; a hop's answer is disclosed after the step citing its passage. **ConvFill** (Srinivas et al. 2025) (67k turns): $\mathrm{thoughts}[i]$ (reasoner knowledge chunk) $\rightarrow$ private block, $\mathrm{response}[i]$ $\rightarrow$ public block, <sil> items (38% of phrases) $\rightarrow$ public filler. **DeepMath-103K** (He et al. 2025): R1 solutions split at $</\mathrm{think}>$ into private reasoning (median 1.2k words) and a reader-facing multi-paragraph solution; paragraphs are interleaved into the reasoning by numeric/LaTeX-anchor coverage (a paragraph is disclosed once all its anchors have appeared in the private stream). We also distil 1,550 problems with Qwen3-32B at high reasoning effort through a two-section prompt, plus GPT-OSS-120B and GPT-6 Astra traces for a teacher comparison. The same conversion yields the two baselines of the protocol: *Standard CoT* (all private reasoning first, then the public text) and *no-think* (public text only), trained with the same recipe.

Reinforcement learning on the disclosure policy

Infrastructure. Rollouts come from vLLM (Kwon et al. 2023) engines running an InnerVoice model class (two-expert last layer and gate inside the forward pass); the trainer holds the policy (base + LoRA + experts + gate), recomputes the gate's decisions for each rollout with one forward pass, scores it, and after each update folds LoRA into the weights and hot-reloads the engines. On one 8-GPU node (4 engines, 4 trainer ranks) a step of 32 prompts $\times$ 8 rollouts with an 8k cap takes 3–4 minutes on A100s and about 5 minutes for 32 $\times$ 16 at 12k on H200s.

Objective. GRPO-style group-normalised advantages (Shao et al. 2024) with the policy gradient applied to the sum of token log-probabilities and gate-decision log-probabilities (weight w_ρ), token-mean normalised, plus a KL penalty (k3 estimator) to a frozen copy of the SFT policy on both tokens (β=0.02) and gate decisions (β_ρ=0.5). Truncated rollouts are kept with a flat penalty (soft overlong punishment, -0.5) rather than masked.

Reward. For a rollout with public text P and private text T,

$$R = \mathbb{1}[\mathrm{correct}(P)]\,\big(1 + w_Q\,Q\big) - 0.5\,\mathbb{1}[\mathrm{truncated}] - 0.3\,\min\!\big(1, \tfrac{n_{\mathrm{short}}}{10}\big), \tag{2}$$

where $Q \in [0,1]$ is the mean of three disclosure-quality terms: the fraction of public lines whose numeric anchors were *all* derived earlier in T (an entailment proxy, the same rule used to build the SFT data), $1 - \min(1, n_{\mathrm{short}}/5)$ with n_{short} the number of public blocks shorter than four tokens (gate flicker), and $1 - \min(1, \mathrm{FCO}/1200)$ for early first content; w_Q=0.2. Every shaping term is thus a bounded bonus available only to correct rollouts, so no disclosure pattern can be rewarded on a wrong answer. Section explains how we arrived at this form.

Hash-reproducible sampled routing. The SFT gate is almost deterministic (on Qwen3-4B, 0.4% of response positions have $0.02 < p_t < 0.98$). With argmax routing, REINFORCE on the gate has no exploration, and negative advantages push the few uncertain logits across the decision boundary, where argmax then flips token to token ("flicker"). We instead sample $r_t = \mathbb{1}[p_t > u_t]$ with u_t a deterministic integer hash of (token id, position), so the engine's decision is a proper sample from the gate's distribution and the trainer recomputes the identical labels without any side channel.

Experiments

Protocol

We follow Wei et al. (2026): accuracy is judged on the *public* stream only (Knights & Knaves: every verdict correct; MuSiQue: final answer, exact match with aliases; AIME25: last $\backslash\mathrm{boxed}\{\}$), decoding is greedy or Average@k at T=1.0, and waiting is measured in token indices over the generated sequence: ARI (mean index of public tokens), ABO (mean onset index of public blocks), AIRW (mean private span before each public block) and FCO (onset of the first public block with task content; filler excluded). Rows are the base model in native thinking mode and without thinking, SFT no-think, SFT Standard CoT (think-then-speak) and InnerVoice, all trained on the same converted data. Test sets are 150 examples ($\pm$4 pt) for K&K and MuSiQue and 30 problems $\times$ 4 samples ($\pm$8 pt) or $\times$ 8 ($\pm$5.6 pt) for AIME25 (Balunović et al. 2025), with a 12k new-token budget (the paper uses 32k). Unfinished samples count as wrong. Models: Qwen3-1.7B and Qwen3-4B (Qwen Team 2025); SFT 2,000 steps $\times$ 16 sequences (K&K/MuSiQue), lr 2×10^{-4} (LoRA) / 10^{-4} (experts, gate). Compute and software are listed in Appendix .

Knights & Knaves and MuSiQue (SFT stage)

Table 1 shows the mechanics: InnerVoice emits 2.7–2.9 public blocks per puzzle with AIRW 103–108 tokens versus 175–177 for think-then-speak and $\sim$2,100 for the base model in thinking mode, and on MuSiQue first content arrives at token 40 versus 97 (CoT SFT) and 950–970 (base). Accuracy is preserved relative to CoT SFT, not improved, at 1.7B (greedy K&K 0.52 vs. 0.55; MuSiQue 0.57 vs. 0.59; all within $\pm$4 pt), and at 1.7B the private stream buys nothing over a direct-answer model (no-think 0.58). At 4B thinking helps on K&K and InnerVoice sits between no-think (0.74) and CoT (0.83) at 0.79 with half the inter-update wait; on MuSiQue the three 4B variants tie (0.70–0.72) and InnerVoice is simply the fastest. Unlike the paper, where SxS SFT loses 20–25 points before RL recovers them, our SFT-stage model stays within noise of CoT SFT, presumably because these traces are two orders of magnitude shorter and highly structured. Ablations (LoRA-only experts, a 30% truncated-think mix) are within noise; the type embedding e_{think} changes neither switching (2.69 vs. 2.73 blocks) nor accuracy.

Table 2 tests decode-time control. Capping private tokens per block keeps accuracy but raises the premature-

	Knights & Knaves (150)					MuSiQue (150)				
method	K&K acc	K&K Avg@4	AIRW	FCO	blocks	MuSiQue acc	MuSiQue Avg@4	AIRW	FCO	blocks
Qwen3-1.7B base, thinking mode	0.413	–	2111	2114	0.81	0.408	–	951	970	0.94
Qwen3-1.7B base, no thinking	0.273	–	0	619	1.00	0.272	–	0	16	1.00
Qwen3-4B base, thinking mode	0.333	–	2046	2059	0.81	0.496	–	940	948	0.92
SFT no-think (1.7B)	0.580	0.508	0	1	1.00	0.584	0.600	0	1	1.00
SFT Standard CoT (1.7B)	0.547	0.533	177	270	1.40	0.592	0.566	96	97	1.00
InnerVoice SFT (1.7B)	0.520	0.512	108	202	2.73	0.568	0.542	39	40	2.45
no e_think input embedding (1.7B)	0.540	0.527	106	189	2.69	0.592	0.592	39	39	2.41
SFT no-think (4B)	0.740	0.667	0	1	1.00	0.720	0.694	0	1	1.00
SFT Standard CoT (4B)	0.833	0.828	175	258	1.41	0.704	0.670	97	101	1.06
InnerVoice SFT (4B)	0.793	0.778	103	177	2.85	0.712	0.686	39	40	2.46

Table 1: SFT-stage results on K&K and MuSiQue (150 test examples each, ± 4 pt). acc = greedy accuracy on the public stream (K&K: all verdicts correct); Avg@4 at $T{=}1$. AIRW/FCO in token indices; blocks = public blocks per example.

decode policy	acc	think	AIRW	FCO	TTFC (s)	P50/P95 pause (s)	premature
nothink	0.213	0	0	1	10.12	0.00 / 0.00	0.609
budget-32	0.507	265	28	191	6.23	0.50 / 0.52	0.706
budget-64	0.527	276	49	197	6.49	0.41 / 1.49	0.555
budget-128	0.540	274	75	195	6.24	0.00 / 3.40	0.383
gate	0.520	283	108	202	6.45	0.00 / 5.70	0.272
fixed-64	0.000	309	64	315	9.56	1.48 / 1.48	1.000
stitch-64	0.000	137	27	1	9.12	0.00 / 0.00	1.000

Table 2: Decode-time disclosure policies imposed on the InnerVoice 1.7B SFT model (K&K, greedy). Caps on think tokens per block trade waiting for premature commitments; forced switches the model never saw in training (fixed-64, STITCH-like stitch-64) break it.

commitment rate (0.71 at a 32-token cap vs. 0.27 for the free gate): the gate's own stopping points are the least premature. Policies that force switches the model never saw in training score zero (the forced switch lands mid-sentence and the answer expert continues the thought), and forcing the INNERVOICE model to skip thinking collapses it (0.21 vs. 0.58 for a dedicated no-think model). A buffer-aware policy therefore has to be learned, which motivates the RL stage.

ConvFill (voice-agent turns)

ConvFill has no gold answers, so Table 3 reports an LLM judge (Qwen3-4B, expected value of a 1–5 score given the reference knowledge) and rule-based filler statistics on 150 held-out turns. The judge does not separate variants (faithfulness saturates at 4.96); behaviour does: the gate reproduces the dataset's think-a-chunk / voice-it rhythm (3.05 public blocks per turn), has the lowest filler rate (0.24 vs. 0.33–0.34) and the earliest simulated first content (4.8 s vs. 5.1–5.7 s; 13.6 s when the same model is forced to speak without thinking, because it rambles).

DeepMath → AIME25 (SFT stage)

On AIME25 thinking is necessary (no-think ≈ 0) and INNERVOICE beats think-then-speak at both sizes (Table 4: 0.175 vs. 0.075 at 1.7B, 0.233 vs. 0.183 at 4B) with first content at token 688 / 303 instead of 3,221 / 1,536 and 4.3–4.7 public blocks instead of 1.7. Two caveats. First, INNERVOICE thinks about twice as long (6.9k / 5.6k private tokens vs. 3.2k / 1.4k) and 17–22% of its samples hit the cap; a budget-matched control at 1.7B gives 0.100 vs. 0.075 (4k cap), 0.117 vs. 0.042 (8k) and 0.175 vs. 0.075 (12k): INNERVOICE stays ahead at every cap, but its advantage grows with the budget because it keeps reasoning while the CoT student stops on its own at $\sim$3k tokens. Second, both are far below the base model's thinking mode (0.258 / 0.342), which however leaves 62–66% of samples unfinished at 12k and is silent for 10k tokens—the paper's SFT-stage picture (SxS SFT 38.3 vs. base 66.3) at our scale.

Table 5 varies the teacher. Private-reasoning *length* is what transfers: Qwen3-32B at default effort (374 words) gives 0.025–0.042, at high effort (1,230 words) 0.192 / 0.208 from only 1,550 problems, matching 11,684 R1 trajectories; GPT-6 Astra, whose reasoning is hidden and replaced by an 85-word explicit section, gives 0.000. INNERVOICE is at least as good as think-then-speak for every long-trace teacher, and the CoT students consistently learn to think *less* than the INNERVOICE students trained on identical traces (e.g. 2,285 vs. 4,745 tokens), which we read as a hypothesis rather than a finding: interleaved disclosure keeps the unfinished parts of the answer in view, so the model keeps reasoning for them.

Reinforcement learning (Qwen3-4B)

Starting from the Qwen3-32B-high INNERVOICE SFT model retrained without e_{think} (its five step-0 evaluations spread

model / policy	faithful.	coher.	filler rate	blocks/turn	think	FCO	TTFC (s)
InnerVoice SFT, router	4.96	4.18	**0.24**	3.05	41	69	**4.8**
InnerVoice SFT, forced no-think	4.98	4.08	0.49	1.00	0	74	13.6
SFT Standard CoT (think-then-speak)	4.96	4.20	0.33	1.01	27	55	5.1
SFT no-think	4.96	4.17	0.34	1.00	0	38	5.7

Table 3: ConvFill (150 held-out voice-agent turns, 1.7B). Judge scores (1–5, Qwen3-4B judge) tie; behaviour differs: the router reproduces the think-a-chunk / voice-it rhythm with the lowest filler rate and earliest simulated first content (0.03 s per think token, 0.35 s per spoken word).

method	Avg@4	unfinished	AIRW	FCO	think tok	blocks
Qwen3-1.7B base, thinking (12k cap)	0.258	0.66	10215	10216	10215	0.34
DeepMath SFT no-think (1.7B)	0.000	0.00	0	4	0	1.00
DeepMath SFT Standard CoT (1.7B)	0.075	0.05	2294	3221	3230	1.71
DeepMath InnerVoice SFT (1.7B)	0.175	0.22	1655	688	6950	4.32
Qwen3-4B base, thinking (12k cap)	0.342	0.62	10300	10301	10300	0.38
DeepMath SFT no-think (4B)	0.042	0.00	0	12	0	1.00
DeepMath SFT Standard CoT (4B)	0.183	0.02	1284	1536	1407	1.73
DeepMath InnerVoice SFT (4B)	0.233	0.17	1336	303	5636	4.74

Table 4: DeepMath-R1 SFT evaluated on AIME25 (30 problems $\times$ 4 samples, T=1, 12k-token cap; $\pm$8 pt). Unfinished samples (no final answer within the cap) count as wrong.

0.142–0.192, the $\pm$8 pt noise in action), Table 6 tracks AIME25 through five reward designs; the mechanism behind R1–R4 is in Section . R5 is the only design whose disclosure profile stays intact through training. Its small run (32 prompts $\times$ 8 rollouts, 8k cap, 300 steps on A100s) ends at 0.367 Average@4, its large run (32 $\times$ 16, 12k cap, 480 steps on H200s) at 0.408, rising from 0.183 at step 0 and staying in a 0.31–0.38 band from step 150. Table 7 re-evaluates both final checkpoints at Average@8: 0.325 and 0.404. The 480-step model answers from token $\sim$470 in about ten public blocks, versus the base thinking mode's single block after 10,301 tokens, at the cost of longer private reasoning (8.3k tokens; 41% of samples hit the 12k cap). Among samples that finish, its accuracy is 0.69 (the base model's is 0.89 on the 38% it finishes); a 32k-budget evaluation matching the paper is [PLACEHOLDER: 32k-budget Average@8 for base / SFT / RL checkpoints; expected to raise all rows, run pending].

Comparison with Wei et al. (2026)

The paper reports, for Qwen3-4B on AIME25 at a 32k budget: base 66.3 (AIRW 17,339), Standard CoT SFT 62.9, Standard CoT RL 73.8, SxS SFT 38.3 (AIRW 4,035), SxS RL 80.0 (AIRW 8,519), with GRPO on DAPO-17K at group 16, batch 32, 480+120 steps. Our numbers are not directly comparable—12k budget, 1,550 distilled SFT trajectories instead of a large multi-source corpus, LoRA plus a two-expert last layer instead of full fine-tuning, 480 RL steps on 8,918 DeepMath problems—but the shape agrees: SFT drops the model below its thinking mode, RL brings it back above, and RL roughly doubles the private reasoning per public block (our AIRW 563 $\rightarrow$ 1,232). Our SFT$\rightarrow$RL gain is +22 points against the paper's +42.

What Breaks Disclosure RL, and Why

All runs in Table 6 share the trainer; only the reward and the prompt pool differ, which makes their failure modes informative.

R1: speak-everything. Correctness plus a first-content bonus ($0.1 \cdot \mathrm{FCO}/600$, paid when correct) on the 1,550 SFT problems. Group accuracy was 0.84, so in most groups the *only* in-group reward variance was the FCO term; the policy learned to open a public block at token $\sim$20 and, by step 100, to carry the reasoning in the public stream (94 public blocks, FCO 21, 52% unfinished). Accuracy rose to 0.292, but the model was no longer disclosing conclusions—it was narrating.

R2: dense penalties without an anchor (run on the larger node, not in the table): penalising unsupported public lines (0.5), short blocks (0.05 each) and FCO with no KL term drove the policy first into word-level think/speak oscillation (step 50: 30 blocks, 21 short) and then into a 70-token private restatement followed by one short public paragraph (AIME 0.19 $\rightarrow$ 0.00 by step 100). Dense shaping terms are always on and consistent; correctness is sparse and noisy; GRPO optimises the former first.

R3/R4: flicker. Making every shaping term a bonus gated on correctness, keeping truncated rollouts with a flat penalty, adding the KL anchor and moving to unseen problems gave clean training for 50–70 steps (R4 step 50: 0.333 with 4.4

teacher traces (think length)	n	model	InnerVoice SFT			Standard CoT SFT		
			Avg@4	think	FCO	Avg@4	think	FCO
DeepMath R1 solutions (dataset-provided; think median 1,237 words)	11684	1.7B	0.175	6950	688	0.075	3230	3221
DeepMath R1 solutions (dataset-provided; think median 1,237 words)	11684	4B	0.233	5636	303	0.183	1407	1536
Qwen3-32B, default effort (think median 374 words)	2287	1.7B	0.025	1343	167	0.017	418	419
Qwen3-32B, default effort (think median 374 words)	2287	4B	0.042	854	145	0.050	300	312
Qwen3-32B, reasoning_effort=high (think median 1,230 words)	1550	1.7B	0.192	4745	1298	0.033	2285	2208
Qwen3-32B, reasoning_effort=high (think median 1,230 words)	1550	4B	0.208	3570	614	0.083	1074	1047
GPT-OSS-120B, reasoning_effort=high (think median 1,132 words) (paper's teacher)	1312	1.7B	0.083	5673	831	0.058	1788	1730
GPT-OSS-120B, reasoning_effort=high (think median 1,132 words) (paper's teacher)	1312	4B	0.183	4826	637	0.075	1047	1033
GPT-6 Astra, explicit <reasoning> section (median 85 words)	542	4B	0.000	329	181	0.000	304	302

Table 5: Teacher-trace comparison on AIME25 (same split/alignment pipeline and SFT recipe; Avg@4, 12k cap). Long private reasoning in the teacher traces is what transfers; InnerVoice $\geq$ think-then-speak for every long-trace teacher.

reward / run	@0	@50	@100	@150	@200	@250	@300	@350	@400	@450	@480	FCO	blocks	short
R1 (FCO bonus, SFT prompts)	0.158	0.217	0.292	–	–	–	–	–	–	–	–	21	93.8	–
R3 (gated Q, KL, unseen pool)	0.192	0.308	0.283	–	–	–	–	–	–	–	–	351	165.7	111.8
R4 (+flicker pen., router KL 0.5)	0.150	0.333	0.200	–	–	–	–	–	–	–	–	236	104.1	78.1
R5 (+sampled routing), p4 32×8	0.142	0.275	0.242	0.283	0.375	0.342	0.367	–	–	–	–	661	9.8	5.4
R5, Greenland 32×16	0.183	0.317	0.317	0.367	0.317	0.383	0.308	0.375	0.325	0.392	0.408	359	10.2	7.5

Table 6: RL round 3 on Qwen3-4B InnerVoice: AIME25 Average@4 (n=120, $\pm$8 pt) at each evaluation step; the last three columns describe the final evaluated checkpoint (FCO, public blocks, short blocks <4 tokens = router flicker). R1 and R3–R4 end in degenerate disclosure (speak-everything or flicker) despite rising or flat accuracy.

blocks) and then word-level flicker (R4 step 100: 104 blocks, 78 short). Raising the flicker penalty and the gate KL delayed but did not prevent it. The gate is near-deterministic, so with argmax routing the REINFORCE term $\nabla \log \pi(r_t)$ is nonzero only at the few uncertain positions, and a negative advantage pushes exactly those logits toward and across zero, where argmax decoding then alternates with tiny hidden-state changes.

R5. Sampled routing with a reproducible hash turns the gate update into a proper policy gradient and, with w_ρ=0.2, removes the flicker: short blocks per rollout fall from 3.8 to 0.6 over training instead of exploding, and both R5 runs keep 8–12 public blocks and FCO 400–900 on AIME25 to the end.

A note on gradient checkpointing. While adding sampled routing we found that our training forward cleared the teacher-forced gate decisions before backward, so under gradient checkpointing the last layer's recomputation used the gate's own decision rather than the label on the $\sim$1% of positions where they disagree. All 4B SFT models were trained with checkpointing and carry this small bias; the RL runs after the fix are unaffected. [PLACEHOLDER: re-run of the 4B SFT without the bug and its AIME25 number].

Limitations

Sample sizes are small (150 test examples; 30 AIME problems $\times$ 4–8 samples), single seed, and we report differences within noise as ties. Our budget (12k) is below the paper's (32k) and the base thinking mode is penalised most by it. The entailment proxy for "unsupported disclosure" only checks numeric and LaTeX anchors. The playback simulator uses fixed per-token and per-word costs rather than a TTS engine; [PLACEHOLDER: end-to-end latency with a real streaming TTS and user study]. We did not evaluate GPQA (gated; Wei et al. 2026), and the voice-stage rewards that use the buffer features b_t (stall-aware pacing) are implemented but untrained: [PLACEHOLDER: buffer-aware RL results]. Finally, InnerVoice inherits every risk of the base model; the readable private stream makes its reasoning auditable but is not a safety mechanism.

Conclusion

A per-token disclosure gate over a two-expert last layer is enough to make a pretrained decoder think while it talks: it is trainable from converted existing data, runs in a standard inference engine, matches think-then-speak fine-tuning where reasoning does not matter and beats it where it does, and—once the gate is trained with a reward that cannot be gamed by wrong answers, a KL anchor and genuinely sampled decisions—RL lifts a 4B model above its own native thinking mode on AIME25 while speaking after a few hundred tokens instead of ten thousand.

model	acc	unfin.	FCO	AIRW	think	blocks	short
Qwen3-4B base, thinking mode (Avg@4)	0.342	0.62	10301	10300	10300	0.4	–
Standard CoT SFT (Avg@4)	0.083	0.01	1047	922	1074	1.6	–
InnerVoice SFT, e_think variant (Avg@4)	0.208	0.01	614	1175	3570	3.0	–
InnerVoice RL R5, p4 (300 steps, 32×8, 8k)	0.325	0.24	669	966	6865	9.4	5.1
InnerVoice RL R5, Greenland (480 steps, 32×16, 12k)	0.404	0.41	472	1232	8333	10.4	7.5

Table 7: Final RL checkpoints on AIME25 at **Average@8** (30 problems × 8 samples, n=240, ±5.6 pt; 12k cap), with the SFT and base rows (Avg@4, n=120) for reference. Unfinished samples count as wrong.

References

Balunović, M.; Dekoninck, J.; Petrov, I.; Jovanović, N.; and Vechev, M. 2025. MathArena: Evaluating LLMs on Uncontaminated Math Competitions. ArXiv:2505.23281.

DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. ArXiv:2501.12948.

Fedus, W.; Zoph, B.; and Shazeer, N. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 23(120): 1–39.

Hao, S.; Sukhbaatar, S.; Su, D.; Li, X.; Hu, Z.; Weston, J.; and Tian, Y. 2024. Training Large Language Models to Reason in a Continuous Latent Space. ArXiv:2412.06769.

He, Z.; et al. 2025. DeepMath-103K: A Large-Scale, Challenging, Decontaminated, and Verifiable Mathematical Dataset for Advancing Reasoning. ArXiv:2504.11456.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.

Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J. E.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

Qwen Team. 2025. Qwen3 Technical Report. ArXiv:2505.09388.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. ArXiv:2402.03300.

Srinivas, V.; Englhardt, Z.; Iyer, V.; and Patel, S. 2025. Thinking While Speaking: Inference-Time Knowledge Transfer for Responsive and Intelligent Conversational Voice Agents. https://convfill.cs.washington.edu/; dataset https://huggingface.co/datasets/zenglhardt/convfill-dataset.

Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*, 10: 539–554.

Wei, J.; Guo, X.; Yu, P.; Zhang, X.; and Ouyang, W. 2026. When to Think, When to Speak: Learning Disclosure Policies for LLM Reasoning. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *PMLR*. ArXiv:2605.03314.

Williams, R. J. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8: 229–256.

Xie, C.; Huang, Y.; Zhang, C.; Yu, D.; Chen, X.; Lin, B. Y.; Li, B.; Ghazi, B.; and Kumar, R. 2024. On Memorization of Large Language Models in Logical Reasoning. ArXiv:2410.23123.

Xie, R.; et al. 2025. Interleaved Reasoning for Large Language Models via Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2505.19640.

Yu, Q.; et al. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. ArXiv:2503.14476.

Appendix

Hyperparameters

SFT: LoRA rank 16, α=32 on all attention and MLP projections; last-layer experts fully fine-tuned; gate hidden 256, output bias initialised to -1 (answer); loss weight λ=1, switch positions ×5; AdamW, lr 2×10^{-4} (LoRA) / 10^{-4} (experts, gate), bf16, context 4,096 (DeepMath) / 2,048 (others); K&K+MuSiQue 2,000 steps × 16 sequences; DeepMath-R1 1,000 (1.7B) / 800 (4B) steps; Qwen3-32B-high 500 steps × 16 sequences at 4B. RL (R5): 32 prompts × 8 (A100) or 16 (H200) rollouts per step, T=1, cap 8k / 12k, group-normalised advantages, lr 10^{-5} (LoRA) / 5×10^{-5} (experts, gate), grad-norm clip 1.0, w_ρ=0.2, β=0.02, β_ρ=0.5, w_Q=0.2, FCO reference 1,200, truncation penalty 0.5, flicker penalty 0.3, prompt pool 8,918 DeepMath problems of difficulty ≥5 not used in SFT; AIME25 Average@4 every 50 steps through the engines.

Compute and software

SFT and HF-decode evaluation: single NVIDIA A100-80GB per run. RL: one 8×A100-80GB node (4 vLLM 0.9.1 engines + 4 trainer ranks) and one 8×H200 node; PyTorch 2.7/2.9, transformers 4.52/4.57, PEFT 0.15/0.21, vLLM 0.9.1 (custom model class and worker extension for in-place weight reload). Teacher traces via Amazon Bedrock (Qwen3-32B, GPT-OSS-120B, GPT-6 Astra). One RL step of 32 × 8 at 8k takes 3–4 min (rollout 100 s, label pass 20 s, update 90 s, weight sync 20 s).

Phase 1: recurrent latent reasoning (null result)

Before InnerVoice we tested whether private reasoning could be replaced by K recurrent passes over M=8 latent

slots (an adapter with cross-attention to the context, or the LM itself as the recurrence, Coconut-style), feeding the slots as soft tokens before the answer. On three self-generated verifiable task families (arithmetic chains, templated word problems, DAG reachability) with Qwen3-1.7B/4B and matched training budgets, every $K \in \{0, 1, 2, 4, 8\}$ and five remedies (context-initialised state, mid-layer context, CODI-style self-distillation, verified intermediate-result supervision, latent-only training) landed within seed noise ($\pm$0.8 pt) of a LoRA-only direct-answer baseline (57.1% / 69.1%), while explicit chain-of-thought reached 80.9% / 79.9%. Diagnostics showed the latent state was context-independent (across-example std 0.0007; shuffling slots across examples changed nothing) and that when forced to be context-dependent the adapter could not compute the intermediates (first intermediate decoded at 23%). This is why the private stream in InnerVoice is readable text.

Data examples

[PLACEHOLDER: one K&K, one MuSiQue, one ConvFill and one DeepMath interleaved trajectory, with gate labels].

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this .tex file directly.

For each question (that applies), replace the "Type your response here" text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel claims are
included} {(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel claims are
included} {(yes/partial/no)}
yes
```

Please make sure to:

- Replace ONLY the "Type your response here" text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the \question command or any other lines in this document.

You can \input this .tex file right before \end{document} of your main file or compile it as a stand-alone document. Check the instructions on your conference's website to see if you will be asked to provide this checklist with your paper or separately.

---

1. General Paper Structure

1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) yes

1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) yes

1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) yes

2. Theoretical Contributions

2.1. Does this paper make theoretical contributions? (yes/no) no

If yes, please address the following points:

2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) NA

2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) NA

2.4. Proofs of all novel claims are included (yes/partial/no) NA

2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) NA

2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) NA

2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) NA

2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) NA

3. Dataset Usage

3.1. Does this paper rely on one or more datasets? (yes/no) yes

If yes, please address the following points:

3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) yes

3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) partial

3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) yes

3.5. All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate cita-

tions (yes/no/NA) yes

3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) yes

3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisficing (yes/partial/no/NA) NA

4. Computational Experiments

4.1. Does this paper include computational experiments? (yes/no) yes

If yes, please address the following points:

4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) partial

4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) yes

4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) yes

4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) yes

4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) partial

4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) yes

4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) yes

4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) yes

4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) yes

4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) partial

4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) no

4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) yes